



masai®

REVISION of specific Topics and Assignment Practice

(TA session revision from 12th Jan to 18th Jan)

Abhishriya, TA



masai[®]

LLM & NLP : intro and few subtopics

Let's understand : Large Language Model (LLM)

A **Large Language Model (LLM)** is a type of artificial intelligence (AI) designed to understand, process, and generate human-like text by analyzing massive datasets. They are a subset of deep learning, which is a branch of machine learning

An **LLM** is a machine that **guesses the next small piece of text**.

It does **only one thing**:

looks at what you wrote

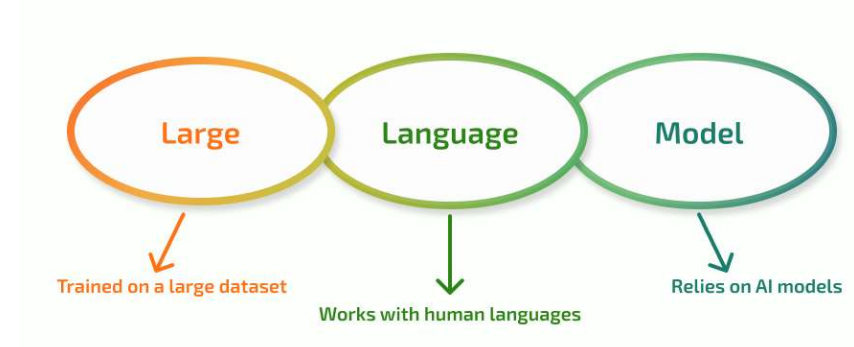
guesses what comes next

That's it.

Example: Sentence:

"The cat sat on the"

The model's job is to guess the **next word**.



Popular LLM Models

- **GPT Series (OpenAI)**: Generative Pre-trained Transformer models (GPT-3.5, GPT-4) known for conversation and content generation, powering ChatGPT.
- **Gemini (Google)**: Natively multimodal, handling text, images, code, and video.
- **Claude (Anthropic)**: Focuses on safety and large document analysis.
- **Mistral/Mixtral (Mistral AI)**: Efficient, open-source models.
- **BERT & RoBERTa (Google/Facebook)**: Strong foundation models for understanding language.
- **LLaMA (Meta)**: A family of open-weight models popular for research.

Few topics to understand for LLM

1. Tokenisation (Breaking text into pieces)

What the model really sees

The model **cannot read words**.

It only understands **numbers**.

So first, text is broken into **tokens**.

Example tokenisation:

"The cat sat on the"

→ ["The", " cat", " sat", " on", " the"]

→ [464, 3797, 3332, 319, 262]

2. Embedding (Turning numbers into meaning)

Token IDs are still just numbers.

So each token ID is converted into a **vector** (a list of numbers).

Think:

"cat" → [0.21, -1.3, 0.88, ...]

"dog" → [0.22, -1.2, 0.90, ...]

Similar words → similar vectors.

This is how the model knows:

cat is closer to dog

cat is far from airplane

3. Transformer (Paying attention)

Now the Transformer looks at **all tokens together**.

Sentence again:

The cat sat on the

When guessing the next word, the model asks:

- What is the subject? → **cat**

- What is happening? → **sat**

- Where? → **on the**

So it gives **more attention** to important words.

This is called **self-attention**.

Simple idea:

- Every word looks at every other word

- Decides what matters most

4. Logits (Raw guesses)

Now the model guesses **all possible next tokens**.

For example:

" mat" → score 12.3

" floor" → score 8.1

These scores are called **logits**.

They represent the model's "internal confidence" or

"guess" regarding which token from its vocabulary should come next in a sequence.

5. Softmax (Turning scores into probabilities)

Logits are messy numbers.

Softmax converts them into **percentages**.

Example:

" mat" → 85%

" floor" → 10%

" roof" → 3%

"Now the model knows:

•“mat” is the best answer

6. Sampling (Choosing the word)

sampling is the core process of choosing the next word (token) from a probability list, using strategies like [Top-K](#), [Top-P](#), and **Temperature** to balance **predictability with creativity, preventing repetitive or nonsensical text** by introducing controlled randomness or focusing choices. It's how the model moves from "The cat sat on the..." to "mat," "couch," or "floor" by selecting from statistically probable options.

7. Autoregressive Loop (Repeat again and again)

After choosing “mat”, the sentence becomes:

The cat sat on the mat

Then the whole process repeats to guess the **next** token.

One token at a time.

Forever.

Like autocomplete.

One-Line Mental Model

LLMs are **very powerful next-token guessing machines** that:

- 1.break text into tokens
- 2.turn tokens into vectors
- 3.use attention to see relationships
- 4.predict probabilities
- 5.pick the next token
- 6.repeat

Let's understand : NLP

1. NLP Basics — What Is NLP?

What is NLP really?

Natural Language Processing (NLP) is the field that tries to make computers work with **human language**.

Humans communicate using:

- words
- sentences
- grammar
- meaning
- Context

Computers understand:

- numbers
- patterns in numbers

So NLP exists to **bridge this gap**.

Without NLP:

- the computer doesn't know "happy" is an emotion
- it doesn't know sentence structure
- it doesn't know intent

NLP converts language into a form machines can learn from.

NLP: Real-World Examples



Virtual Assistants



AI Chatbots



Language Translation



Voice-to-Text Applications

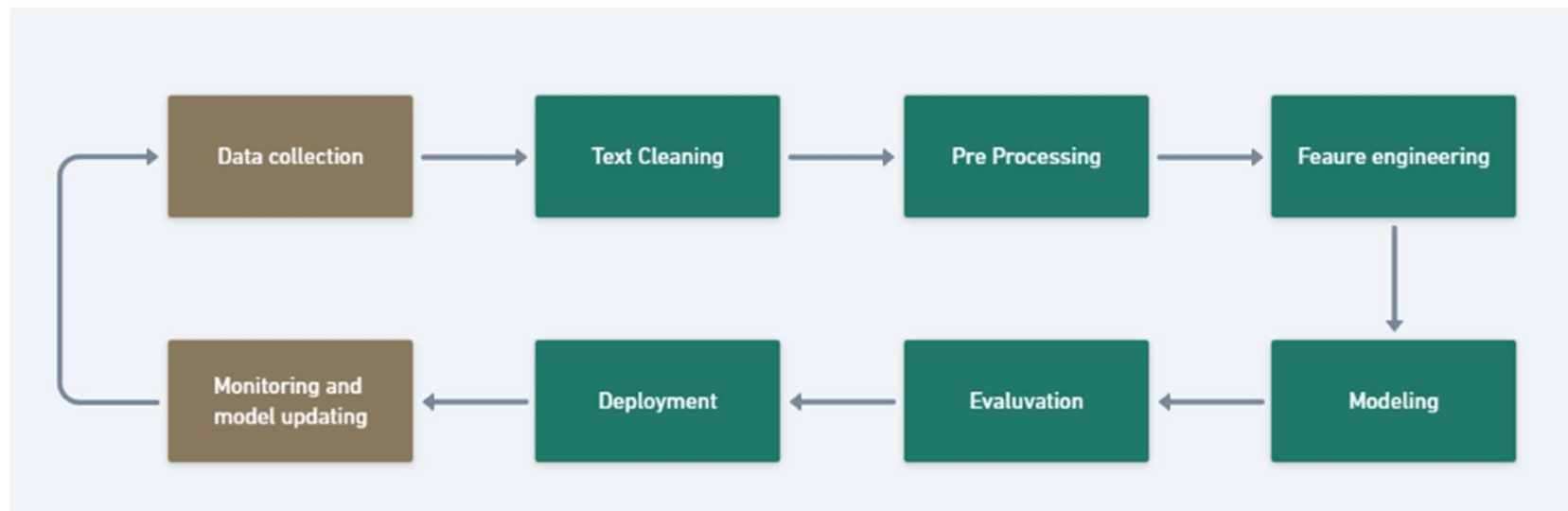


Email Spam Filtering



Text Summarization and Generation

Let's understand : NLP Pipeline



2. Text Processing — Cleaning Text for Machines

Why raw text is a problem

Humans are flexible:

- “Hello”
- “hello”
- “HELLO!!”

All mean the same.

Computers are **strict**:

- “Hello” ≠ “hello”
- “Al.” ≠ “Al”

So we **clean text** first.

Common text processing steps (why each exists)

1. Lowercasing

Makes “Apple” and “apple” the same.

2. Removing punctuation

“good!” and “good” should match.

3. Removing extra spaces

Machines treat spaces seriously.

4. Removing stopwords (optional)

Words like “is”, “the”, “and”

They appear everywhere and add little meaning.

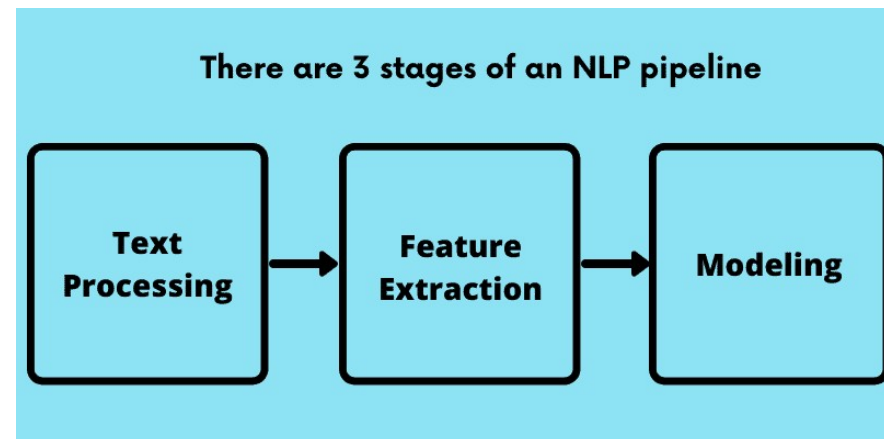
5. Stemming / Lemmatization (optional)

“running”, “runs”, “ran” → “run”

So meaning stays same.

Important:

- Traditional NLP needed heavy cleaning
- Modern LLMs do **less cleaning** because tokenizers handle complexity



3. Vectorization — Turning Text into Numbers

Why numbers are mandatory

Neural networks **cannot process words**.

They only process:

- vectors
- matrices
- numbers

So every piece of text must become a **number representation**.

This is vectorization.

TF-IDF — smarter counting

Idea:

- Words common **everywhere** are less useful
- Words unique to a document are more useful

Example:

- “the” appears in all documents → low importance
- “neuron” appears rarely → high importance

TF-IDF gives **weights**, not just counts.

Still:

- No real meaning
- No understanding of similarity

4. Tokenizer — Breaking Text into Pieces

What a tokenizer really does

A tokenizer:

- splits text into **tokens**
- assigns each token a **number**

*So **how you tokenize = how the model understands language**.*

Wrong tokenizer = nonsense output.

5. Embeddings — Giving Meaning to Tokens

What embeddings actually are

An **embedding** is a vector that represents **meaning**.

Not spelling.

Not appearance.

Meaning.

Each word or sentence becomes a point in a high-dimensional space.

Why embeddings are powerful

They allow:

- similarity search
- clustering
- semantic understanding
- reasoning patterns

This is the **core intelligence layer**.

6. Similarity Search — Meaning over Words

Keyword search vs similarity search

Keyword search:

- “AI learning”
- matches exact words

Similarity search:

- understands meaning

Example:

Query:

“How to study artificial intelligence”

Matches:

“Best ways to start machine learning”

Words differ.

Meaning matches.

How similarity search works conceptually

1. Convert text → embedding
2. Compare embeddings using distance
3. Smaller distance = closer meaning

This is how:

- search engines
- recommendation systems
- RAG systems work.

7. Conceptual End-to-End Flow

Think of the pipeline like this:

1. User types text
2. Text → tokens
3. Tokens → embeddings
4. Embeddings → similarity / attention
5. Model predicts next tokens
6. Tokens → text

This is **everything** modern AI text systems do.

How LLMs extend NLP ideas

Traditional NLP:

- small datasets
- small models
- task-specific

LLMs:

- huge datasets
- huge models
- one model does many tasks

Same foundations.

Much larger scale.

How NLP Foundations Become a Transformer

Full Pipeline (Bird's-Eye View)

When you type a sentence:

- 1.Text processing
- 2.Tokenization
- 3.Token IDs
- 4.Embeddings
- 5.Transformer layers**
- 6.Logits (scores)
- 7.Probabilities
- 8.Next token

Transformer is **step 5**.

End-to-End Transformer Flow

You write text.

- 1.Text is cleaned if needed.
- 2.Text is split into tokens.
- 3.Each token is converted into a number.
- 4.Numbers are converted into embeddings that carry meaning.
- 5.All embeddings enter the Transformer together.
- 6.Each token looks at all other tokens using self-attention.
- 7.Important words influence each other more.
- 8.This process repeats across many Transformer layers.
- 9.The final layer produces scores for possible next tokens.
- 10.Scores are turned into probabilities.
- 11.One token is selected as the next output.
- 12.The new token is added to the text.
- 13.The process repeats until the response is complete.

Step 1: Tokens Enter the Model

Input text:

“The cat sat on the”

Tokenizer breaks it into small pieces called tokens:

“The”, “ cat”, “ sat”, “ on”, “ the”

Each token is replaced by a number:

464, 3797, 3332, 319, 262

Important point:

The Transformer **never sees words**.

It only sees these numbers.

Step 2: Embeddings Enter the Transformer

Each token number is converted into a **vector**.

So instead of:

464

The model sees something like:

a list of numbers that represents meaning.

Now the input is:

a sequence of vectors, one for each token.

Still no text.

Only numbers with meaning.

Step 3: Why Transformers Exist

Older models (RNNs):

- read one word at a time
- were slow
- forgot early words in long sentences

Transformers fix this by:

- looking at **all tokens together**
- not one by one

This makes them fast and good at long context.

Step 4: Self-Attention (Core Idea)

Self-attention means:

Each token decides

which other tokens are important.

In the sentence:

“The cat sat on the”

When guessing the next word:

- “cat” matters
- “sat” matters
- “the” matters less

The model learns these importance levels automatically.

Step 5: Tokens Share Information

Using self-attention:

- “sat” pulls information from “cat”
- meaning flows between tokens

This is how the model understands relationships.

Step 6: Multiple Transformer Layers

This attention process happens:

- many times
- across many layers

Each layer improves understanding.

Early layers learn structure.

Later layers learn meaning and patterns.

Step 7: Ready to Predict

After the final layer:

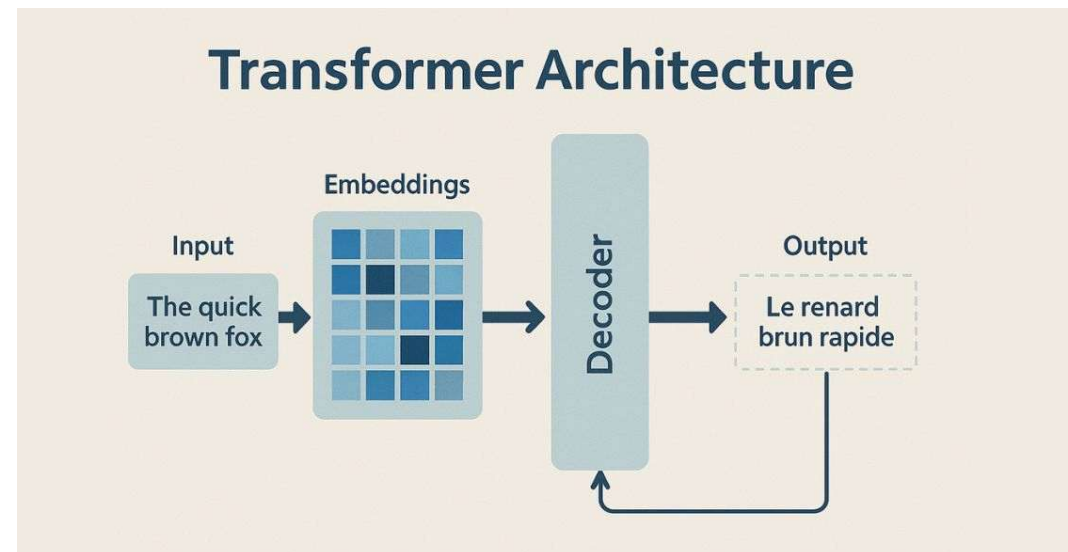
- the model has a strong understanding of the sentence
- it is ready to predict the next token

Example guess:

“ mat”

One-line summary

Transformers turn tokens into vectors, let all tokens look at each other using attention, and then predict what comes next.





masai®

Assignment Question Practice

PART 2: REAL INTERVIEW QUESTIONS

(IBM / ML Engineer / Research roles)

- Why does a Transformer work with token numbers instead of words?
- What problem does tokenization solve for unknown or new words?
- Why are embeddings used instead of raw token IDs?
- What does it mean when two words have similar embeddings?
- Why were Transformers introduced instead of RNNs?
- How does processing all tokens at once help Transformers?
- What information does a Transformer receive as input?
- Why are multiple Transformer layers stacked instead of using just one?